Names of contributors: _____

NIA(s): _____ Group: _____

**P1PP: Practice 1 Pre-processor**

You have to implement in c a program that executes few functionalities of a c preprocessor .

The c pre-processor is a text substitution tool. All pre-processor commands, called directives, begin with a # symbol. This symbol must be the first nonblank character of the line, but for readability purposes it should begin in the first column. You can read the functionality of a c pre-processor in the online tutorial in [4] or any other you like. In this practice we will implement few of these functionalities:

1. Directives
    a. #include
    b. #define
    c. #ifdef - #endif (a reduced version)
2. (optional) Parametrized macros: A macro is like a function but at the preprocessing level. It is a substitution function that has parameters to provide different situations or values to the substitutions. We implement a reduced version.
3. Eliminate comments and replace each of them by a space (empty lines are not eliminated)

The program should have different options to run to apply different levels of preprocessing. Hence it has to interpret the following flags (as arguments of the program) as indicated:

1. –c it should eliminate the comments (default)
2. –d it should replace all directive starting with #
3. –all it should do all processing implemented
4. –help: it prints your "man page" of your pre-processor

The option of only comments is assumed to be the default option when no flags are indicated. All flags or a subset of flags (or any) should be handled. Therefore, "-c –d" should do the same (with the current implementation) as "-all". Also, "-c –d –all" should be correct syntax for asking to do all. In addition, the order of the flags is not relevant, hence "-c –d" should be the same as "-d –c". Warning: These options should not drive the structure of your program. Therefore, start your design and program without these options, and assume that only the -all have to be implemented. And once you have the program designed, add the possibility to just execute part of the options.

The user information of the pre-processor should be documented in a form of a "man page" that should be showed in the display when the –help option is requested. If the –help flag is included it neglects any other flag indicated and just prints the help page (but it does not execute the preprocessing).

The pre-processor has to process a .c file that contains the main.

The usage of the program shoud be: preprocessor <flags> <program.c> where *flags* should be the list of flags as explained above and *program.c* should be the file to process.

Assume the input example program provided with the handout material input-example. As example, options applied to this example are:

- preprocessor input-example.c or preprocessor input-example.h
    - The output is the same as the input file but without the comments in it.
- preprocessor –all input-example.c (also: preprocessor –d –c input-example.c, or preprocessor –c –d input-example.c)

  - This replaces all directives inside the file, and hence the input-example.h is included in the output. There should not be any comment in the output file.
- preprocessor –help: it should print the man page of your preprocessor
- preprocessor –d input-example.c
  - This processes all directives of input-example.c (and hence it will include the code of input-example.h). However, the resulting file keeps all comments as the original ones.

The pre-processed file should be called the same but adding the suffix _pp, this is, the pre-processed of input-example.c should be called input-example_pp.c, and the pre-processed of input-example.h should be called input-example_pp.h.

The design of the implementation is very important. We build a reduced version of a pre-processor. But it should be done as an ENGINE that eventually it can be added all necessary functionality. So the design has to include a data structure and conceptual design that generalizes how to do things, and it should not be a particular hard-wire code that only works for the particular reduced version of this specification. More description is given below. In addition to the man page we want a design/development manual to document in detail all functionalities implemented. In fact, the pre-submissions (described in the submissions section 2) intend to track the design progress. IMPORTANT: the design should be structured differently than the list of things to be done. So the list of things to be done is to make sure that someone things about each one, but it is not the split of how the code should be structured, and hence how the code should be divided and worked out between the different members of the team. Build your design and assign then the implementation work.

The usability is also very important and the user must have informative messages and manual on how to use it, and also clear indications on the errors so that the user can quickly interpret the errors and decide how to fix them (so it should print all information the program has on variables relevant to the error).

We want to impose good practices on documentation as compilers are big pieces of software that many people are involved, and they live long time and many updates. We want documentation at three levels (see requirements document for a summary of it):

1. Information to users: user's manual (man pages). Remember that the user of a preprocessor is a programmer.
2. Conceptual design slides: it describes the design decisions and the structure of the program to be built (ideally graphically), detailing the data structures to be created and the corresponding functions to manipulate each of them (the corresponding library for each data structure) with the explanations and justifications to know how the program works globally, so that each one can work independently a portion of the design. The design has to define portions with the interaction with the rest. It has to include also the scope of each functionality indicating what it covers, and what it is left out, and hence also describing the situations of errors that should handle and how. So it should detail the exact description of the data structures to be used and exact functions (with function names, parameters with exact types, output and functionality to be produced).
3. Well documented code: The developer has to have enough documentation inside the code. So we require: 1) A header comment on top of each code file. This header has to explain what overall functionality provide the whole set of structures (for .h) or functions (for .c) included in the file. It can add extra informative explanations like date of creation and a sequence of modifications and reasons, authors and other relevant information you consider. 2) A header comment in each function to explain what the function conceptually does, what each parameter conceptually means, what is the meaning of the conceptual return value. It is important to indicate properties or specific algorithm names it implements. 3) a conceptual comment in each declaration of variable or definition of data extructure (except temporal variables) and 4) Comments inside the functions: These can be individual comments to individual lines of code difficult to interpret conceptually, or a comment that explains conceptually a group of lines of code so that to interpret what the group intends to do conceptually. 5) Remember that comments have to provide conceptual

understanding and not literal interpretation of the code. These comments have to be useful to the programmers looking at the code and they know c. Unless you use a c instruction that you assume the programmer don't know, the comments cannot be a literal description of what the instruction does. A comment that just says what this c instruction does as in a manual (rephrasing the code literally with natural language) fills the display with more text for nothing. So they become a problem more than a help. It provides too much unnecessary reading and the real useful conceptual comments are very difficult to identify. 6) Remember that comments need to be updated if the code changes. They have to be consistent with the current version of what the code does.

The program should provide always an informative error message (to the display, and we can redirect display to a file when we want, in the command line) if something goes wrong. The errors must indicate the line of the input file where the error occurs (see __LINE__ macro in the pre-processor definition [1]).

Ideally, the program should never break for not having a correct code file, instead it must detect the non-complience and take a decision, and one possibility is to finish the program correctly providing informative information to the programmer to take action and correct the mistake. Finishing the process must be the last option in front of an error. Decide to neglect the word or line or group of lines and continue the processing of the input file as much as possible so that to provide the maximum information (errors messages) on the current execution. The submission must guarantee that there is at least one error situation implemented fully in this way. It is not required to implement all possible error messages. However, in the documentation, list as many situations not handled as you can identify so it is well documented what it is not supported (not implemented) yet. Consider that eventually these situations would need to be implemented. So do not do an error handling just assuming just one error, but generalize the implementation assuming that many errors would eventually need to be handled in a future extension of the code.

Think how is the best way to implement and document the messages inside the code, as notifications to the user, and in a user's manual (final report). Take into account that all phases of the compilation have error messages, so think of a separate module that could be reused or extended in potentially next steps of a compiler. Remember you are not compiling c and identifying the errors of the c code. You just have to deal at this point with the preprocessing functionaltities (and potential error messages) you implement. The requested pre-processor has a very simplified scope, but think this should be the starting point of a full development with a huge number of situations (correct or incorrect) to consider either in the same pre-processor, or to use this error handling in different phases of a compiler (regardless if we implement them or not).

The program must take a text file (in principle a c file, hence .c or .h, but it should process any text file) and generate an output file with the same (c code) text preprocessed as indicated above. The resulting file name should follow the specification above, add the suffix _pp, this is, the preprocessed of input-example.c should be called input-example_pp.c.

Your pre-processor should be able to work well with the given program input-example. It should also work well with your own (submitted) code. So your pre-processor program will be taken itself as example to run it, and see if it can handle the required pre-processor substitutions included in the code and specified in this handout. Remember that your code already passes the c pre-processor, so it is an example of code with no mistakes. The pre-processor directives not supported should not be a problem as your pre-processor should neglect the non-supporting directives and keep them in the output as in the input. This is to make you aware of where your real program (github repository code) uses the pre-processor, it is key to have parametrized code.

# 1 Syntax of pre-processor

In here we described a simplified syntax of the functionalities the pre-processor must support. Along the course we will study many formalisms to define syntax (and others). But right now we just use a simple description to start with.

Recall that the pre-processor directives are not included in the output line. They are the commands to instruct the pre-processor to decide what to include in the output file. Hence, these lines cannot be passed to the compiler.

You can use any pre-processor manual to get the full description (and examples) of the directives. As example we suggest [4]. But feel free to use your preferred one. Note that we do not implement the full description, but a reduced version as described next. The submission should at least be able to processed correctly the program example provided (input-example.c). The directives not implemented should not be processed but should not abort the execution with and error. This program example is also a program template to start your code with the definition on how to divide the code in different files and get them work as a single code. It is not required to use this template and it is given just for your information if you need and example on how to do it.

## *1.1 Directives*

The pre-processor directives we consider in this assignment is a subset of the full c pre-processor. The parts not covered, if appear in the file to be processed should be neglected and it should not cause a break or termination of the processing. It should be treated as the c code and copied the same from the input to the output- The directives to be considered are the following.

### 1.1.1 #include

Our syntax: `#include "file"`

The includes allow us to divide the code in different files. The directive include substitutes the #include file by the entire content of the file indicated. The file included should be processed also as the input to have includes recursively, and any other functionality to be applied.

The compiler files, such as `stdio.h`, are indicated as `#include <stdio.h>`. It looks for the file in the path. The files included in the same directory as the code are indicated as `#include "myinclude.h"`.

The example code provided with the handout gives an example of a header file and in the corresponding include of program header files and compiler files (stdio.h) are commented out. Your program must be able to include the files of the same code (#include "filename") in the program folder. The solution does not require to handle includes of the compiler (#include <filename>). These includes, if they exist in the input, will not be supported and the directive will remain in the output as is in the input.

### 1.1.2 #define

Our syntax: `#define <str1> <str2>`

For simplicity, we assume a simplified syntax where `str1` can be any string of characters with no spaces, and it becomes an identifier. The first space after the string `#define` decides `str1`, and `str2` are all characters until the end of the line (except the character "\" that it has special meaning, see section below).

Note that c ends all line codes by ";", but the pre-processor does not. Hence if the line ends with ";" this character is the last character and it is included in the substitution.

The pre-processor replaces all occurrances in the file of `str1` by `str2` and eliminates the `#define` from the output file. Note that `str2` may also require substitutions if they include pre-processor directrives.

Example:

```
#define max 3

…
for(i = 0; i < MAX; i++) a += i;
```

after the pre-processor the #define line is not there and the for line becomes:

```
for(i = 0; i < 3; i++) a += i;
```

### 1.1.3 #ifdef - #endif

Our syntax:

```
#ifdef <str>

// anything can be in here
// (the pre-processor does not interpret this part)

#endif
```

The <str> is an identifier previously defined with `#define`.

Between the lines `#ifdef` to `#endif` there is valid code that we only want to use if the identifier `str` is previously defined with a `#define`. So the pre-processor has to look for the table that includes all defined idnetifiers and if it finds `str`, the following code is copied to the output, otherwise, it is not copied (eliminated in the output version).

For simplicity, `str` can only be just one single identifier defined with a #define directive. Hence, the `str` syntax has to follow the same syntax as `str1` in `#define` directive.

Example:

```
#define DEBUG

#ifdef DEBUG
     printf("this message is only printed if the DEBUG flag is
1");
#endif
```

The pre-processor keeps the lines between ifdef-endif in the output file if the identifier DEBUG is defined. Therefore, the printf line is kept in the output file with the example as is. If the line `#define DEBUG` is eliminated or commented out, then the pre-processor does not copy the printf line to the output, it eliminates the printing line.

The file example provided with the handout includes an example of this.

## *1.2 (Optional) Parametrized Macros*

Our syntax: `#define str1(s1, s2, s3) str2`

The macros are a parametrized substitution simulating a c function but at the pre-processor stage. It is an extended version of the `#define` directive which adds parameters to the definition. It is defined

as a constant identifier `str1` (the same as `str1` in the original `#define` in section 1.1.1) but it is followed by the parameters between parenthesis. Hence the macro starts with an identifier string `str1`, it follows an open parantesis, it follows an identifier string `s1`, it follows a comma and another identifier string (`sn`) as many times as needed before closing the parantesis. Up to here there is the syntax definition of the macro name, which follows the string `str2` which defines the substituting text. The macro `str1(sn..)` is replaced by the text `str2`, and in addition `str2` should have instances of `sn` to be replaced by the corresponding value of the parameter `si` in the macro usaged inside the code. The same as in section 1.1.1, `str2` can be any text and it is the responsibility of the programmer to impose that this is correct c code so that the c compiler can understand it.

Example:
```
#define PRODUCT(x, y) ((x) * (y))

main(){

..
   printf("The product of 3 and 4 is %s\n", PRODUCT(3, 4));
}
```

In the code we can use PRODUCT like a function but the compiler will not see the definition of this function, instead it will see:
```
printf("The product of 3 and 4 is %s\n", ((3) * (4)) );
```

The provided code example it includes an example of WARNING and ERROR formats implemented as pre-processor macros. Make sure your pre-processor can handle them.

A full design and implementation of this functionality counts for an outstanding level of the submission.

### *1.3 Eliminate comments*

There are two ways to put comments in c

- Single line comments using "//". all characters behind // are comments until the end of the same line. The pre-processor in this case must identify the two consecutive characters (with no characters in between) and eliminate them and all following characters until the end of the line. The end of the line should not be eliminated to kept separate the current line and the next.
- Multiple lines comments using with /* */. All text between /* and */ are comments including the end of line. So this comment can have multiple lines because it identifies when it starts and when it finishes. It can also be a short comment at the end of the line of a code, or in the middle of the line. The character immediately after */ counts as code. The pre-processor, in this case, has to identify the two limiters and eliminate all characters between them.

You can see how the c pre-processor works by compiling gcc with –E flag, and it shows intermediate file. Note that the #includes add the code of the included files so it gets a lot of text before your preprocessed code.

## 2 Submissions

You have to work and submit the practice with a practice team of 6 people. See aula global to choose your team. Put in the submission the team identifier (GA, GB, …), but also put the names to confirm that all have contributed in the submission. Just put the names of the members of the team who contributed in the corresponding submission. Inform in advance to the professor if the team is not working well, so we can address any issue, and so she is informed of any particular time constraints with the submissions of

particular students. Otherwise, if the name of a student who has not contributed is included, it will be considered plagiarism, and all team with be accounted responsible.

You have to prepare the material for each team meeting. This assignment has 2 official team meetings (TM1 and TM2 sessions) that, as any team meeting, require to provide material prior to the meeting and hence there are 2 pre-submissions with meeting material and the final (or actual) submission for this practice. See requirements document to see what material to provide for the team meetings.
You are also required after each meeting to submit the individual feedback sheet in aula global and to the presenter's team. See submission's section in aula global.

**Final submission**:
The final submission must include:

1. A zip version of your github repository which must contain the c code, README and also all information requested below.
2. A set of sample input files to test all situations (correct files and incorrect files). Document well the cases inside the file and also in the report.
3. The conceptual design slides used in the team meeting with the complete conceptual view of the project.
4. The user's manual as explained above.
5. The modules's author responsibility history.
6. A status table indicating for each feature requested in the handout (if it is done, started, not working or completed) and if not completed a short comment on what it has been done and why it is not working so there is a clear view of the submitted code.

Have a documentation folder inside the github repository with all this information so that with the zip of the repository it includes all. Name your github repository with the prefix Gx_ so that the zip file contains at the beginning of the file the identifier of the team.

Remember the submissions must be done in aula global before the deadline indicated there. It is not a submission just the access to faculty to your github respository. **Make sure you put the name/group inside the files (reports, code[1], input samples...)** so that the authors can be known when reading the documents and also when listing the files in a directory where all submissions can be collected.

# 3 Evaluation criteria

The practice is evaluated with in 3 levels with the meeting feedback sheets. Make sure you read the meeting feedback sheets to apply the feedback provided. The final practice evaluation is also a feedback sheet with a final practice level.

There are 3 levels: non-competent, competent and excellent/outstanding. The non-competent does not reach the minim level required. The competent has a level to pass, and it means the team has this part of practice with a 5. An outstanding means the submission (or an individual criteria) is of very high quality and gives extra grade to this practice submission.

# 4 Copy and Plagiarism

Remember that all work must be yours. You have to explain things in your own words, and the code has to be written by you. You cannot use copy code from other sources (books, other solutions, or any other), and if you use ideas or material you have to put it in your own words or programming.

[1] Put he authors inside each code file as a documentation header on top of each file, as indicated in the requirements document.

Add a bibliography section in your report and do the correct citations in the sections where you explain the ideas you have used from sources. It is a good practice to use sources and material, so explain it if you use them well. But do not copy from these sources.

Remember that you can discuss issues with other students and teams, but you cannot exchange code or text and used them as is. You can exchange ideas, but once you understand them, the team has to do your own version.

# 5 Materials

1. Input program example input-example code: This is an example of input code that your pre-processor should be able to process.
2. Modules_template project: A template of a project with several modules, that each module can be tested independently with a different main. Code given in the aula global as zip file and also as a github respository link.
3. Compilers Requirements Document. The document that describes the requirements to comply for the elaboration process and hence common to all practices of the course: https://aulaglobal.upf.edu/pluginfile.php/10493789/mod_resource/content/5/2526-COMP-requirements.pdf
4. C tutorial: Pre-processor: https://www.tutorialspoint.com/cprogramming/c_preprocessors.htm#